

Generative AI disclosure use statement from Chase Kindig:

A combination of ChatGPT 6, ChatGPT 5.3, and Gemini Pro-Extended was used to evaluate different pseudocode and functions of dc.py. These were verified with ample testing and referring class reading material. That and a few Stack Overflow forums cause I'm old and these forums are eternal. Also tested prompts against ChatGPT MIL, Gemini 3.8 MIL, and Grok 4.5 MIL. No responses were ever directly referenced from these models, but they were used to explore more of the material while working within work environments with very limited external access. A.k.a. no access to UMBC blackboard or use of any UMBC/personal accounts including email, drive, etc. These models were used with the question prompt #4 as well. Responses could not be shared, but it did help evaluate the model used in question 4.

Question #1 Answers:

1. Steps

1.1. Formula/pseudocode:

$$h_steps(s) = d(s,g)$$

1.2. Unavoidable remaining cost (lower-bounds):

one legal action changes exactly one position, so one action can correct at most one currently mismatched goal position. Therefore at least $d(s,g)$ actions remain.

1.3. Why it cannot overestimate true optimal remaining cost:

Every action costs exactly 1, the remaining optimal cost is at least $d(s,g)$. The heuristic is 0 at the goal and never overestimates.

2. Scrabble

2.1. Formula/pseudocode:

$$h_scrabble(s) = \sum(\text{value}(g[i]) \text{ for each } i \text{ where } s[i] \neq g[i])$$

2.2. Unavoidable remaining cost (lower-bounds):

Inserting the required goal letter at position i costs exactly its assigned Scrabble value. Those insertions are unavoidable for every mismatched position.

2.3. Why it cannot overestimate true optimal remaining cost:

Any detour, including changing the correct position away from the goal and later restoring it, adds nonnegative extra cost. Summing only the unavoidable target-letter insertions therefore cannot exceed the true remaining cost.

3. Frequency

3.1. Formula/pseudocode:

$h_freq(s) = 0$, if $d = 0$

$h_freq(s) = (d-1)(1 + r_min) + (1 + r(g))$, if $d > 0$

3.2. Unavoidable remaining cost (lower-bounds):

At least d actions are required because each action changes one position. The final action must enter the goal, so that action necessarily costs $1 + r(g)$. Each of the other $d-1$ required actions costs at least $1 + r_min$ because all dictionary rarities are nonnegative and r_min is the smallest available value.

3.3. Why it cannot overestimate true optimal remaining cost:

The formula assumes the cheapest possible costs for those unavoidable actions, so it cannot overestimate. It is also 0 at the goal.

Question #2 Answers:

Word pair	Cost	UCS cost	UCS expanded	A* cost	A* expanded	Reduction
WHY -> ASK	Steps	5	773	5	16	97.93%
WHY -> ASK	Scrabble	9	765	9	13	98.30%
WHY -> ASK	Frequency	38.855407	808	38.855407	634	21.53%
BEAR -> DUCK	Steps	4	691	4	6	99.13%
BEAR -> DUCK	Scrabble	11	2981	11	6	99.80%
BEAR -> DUCK	Frequency	38.240228	1084	38.240228	301	72.23%

1. Yes. UCS and A* returned the same optimal cost in all six experiments. The exact returned path can differ when multiple optimal paths exist; the comparison here is based on total optimal cost.

2. For both word pairs, the Scrabble heuristic produced the largest percentage reduction in expanded nodes: 98.30% for WHY -> ASK and 99.80% for BEAR -> DUCK.
3. For both word pairs, the frequency heuristic was least informative by the experiment: only 21.53% reduction for WHY -> ASK and 72.23% for BEAR -> DUCK.
4. The steps heuristic directly counts unavoidable letter corrections. The Scrabble heuristic is stronger because it also prices each unavoidable target-letter insertion using its actual replacement cost. The frequency heuristic can prove only that a minimum number of actions remain, that the final transition must pay the goal rarity, and that earlier transitions cost at least the globally smallest rarity. It cannot predict which high-rarity intermediate words the dictionary connectivity will force, so many states receive similar lower bounds and A* has less guidance.

Optimal Paths:

WHY -> ASK (steps/scrabble): why -> wha -> aha -> ahs -> ass -> ask

WHY -> ASK (frequency): why -> way -> was -> has -> his -> ais -> ass -> ask

BEAR -> DUCK (all three objectives in this run): bear -> beak -> beck -> buck -> duck

Question #3 Answers:

- steps:
Yes, every edge costs 1 in both directions. Reversing any path preserves its total cost, so the two optimal costs are equal.
- Scrabble:
No, the cost is determined by the newly inserted letter. Reversing an edge normally inserts a different letter, which may have a different value.

Counterexample: DOG <-> COG

Both words are in the supplied dictionary and differ only at position 0. DOG -> COG inserts c, which costs 3. COG -> DOG inserts d, which costs 2. Each direct edge is optimal because every path between these adjacent states must eventually perform the required insertion at that differing position, and extra actions have positive cost. Thus the optimal costs are 3 and 2.

- Frequency:

No, The cost is $1 + \text{rarity}(\text{destination})$. Reversing an edge changes the destination, so the edge cost generally changes.

Counterexample: DOG $\leftrightarrow$ COG

$\text{rarity}(\text{dog})=5.819885$ and $\text{rarity}(\text{cog})=11.393166$. Therefore DOG $\rightarrow$ COG costs $1+11.393166 = 12.393166$, while COG $\rightarrow$ DOG costs $1+5.819885 = 6.819885$. For adjacent states, the one-step edge is optimal because every alternative still must enter the destination and would add at least one extra positive-cost action.

Question #4 Answers:

1.)

AI Model: Gemini 3.1 Pro – Extended Thinking

Prompt:

“In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary. The baseline heuristic $h_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal. By “more informative,” we mean that the proposed heuristic should be at least as large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.”

AI’s proposed heuristic:

$$h(n) = \begin{cases} 0 & \text{if } n = g \\ h_0(n) + \text{rarity}(g) & \text{if } n \neq g \end{cases}$$

2.) pseudocode:

```
def h(self, node):  
    state = node.state  
  
    # Count positions that still differ from the goal.  
    h0 = sum(a != b for a, b in zip(state, self.goal))  
  
    # No remaining cost once we are already at the goal.  
    if h0 == 0:  
        return 0  
  
    # Every mismatch needs at least one change, and the final  
    # change must pay the rarity cost of entering the goal.  
    return h0 + dictionary[self.goal]
```

3.)

Is it admissible?: Yes

Let: $d = h_0(n)$

Any path from n to the goal must make at least d letter changes, because one action changes only one letter.

Under frequency cost, every move costs: $1 + \text{rarity}(w)$.

Suppose the path uses k moves. Since $k \geq d$: true cost = $\sum_{i=1}^k (1 + \text{rarity}(w_i))$

The final entered word is the goal, so one of those costs is: $1 + \text{rarity}(g)$

All other rarity values are nonnegative. Therefore: true cost $\geq d + \text{rarity}(g)$

which is exactly the proposed heuristic: $h(n) = h_0(n) + \text{rarity}(g)$

So the heuristic never overestimates the true remaining cost. It also dominates h_0 : $h(n) \geq h_0(n)$ because rarity is nonnegative. For the supplied test goals, the goal rarity is positive, so it is strictly larger than h_0 for every non-goal state.

4.)

No repairs needed, but if a dictionary allowed a goal with rarity exactly 0, then the heuristic would equal h_0 rather than being strictly more informative. It would still be admissible, though.

5.)

Test:

Using the supplied search code and frequency-cost problems:

Problem	Heuristic	Solution cost	Nodes expanded
WHY → ASK	h_0	38.85540652	755
WHY → ASK	Proposed	38.85540652	634
BEAR → DUCK	h_0	38.24022840	720
BEAR → DUCK	Proposed	38.24022840	302

For WHY → ASK, expansions dropped from 755 to 634: $\frac{755-634}{755} \times 100 \approx 16.03\%$

For BEAR → DUCK, expansions dropped from 720 to 302: $\frac{720-302}{720} \times 100 \approx 58.06\%$

Conclusion:

The evidence supports using the proposed heuristic. It is admissible, it is more informative than h_0 for these problems, and it reduced the amount of search work without changing the optimal answer.